16. Софтуерно инженерство и неговото място като дял от знанието. Софтуерен процес и модели на софтуерни процеси. Концепция за многократна употреба
Изложението по въпроса трябва да включва следните по-съществени елементи:
1. Софтуерното инженерство – какво е софтуер, видове софтуер, същност и обхват на софтуерното инженерство.
2. Софтуерен процес – фази и основни дейности, видове модели и езици за моделиране, шаблони за описание.
3. Сравнителен анализ на описателни модели на софтуерен процес – модел на водопада, прототипен модел, модел на бързата разработка, спираловиден модел и др.
4. Концепцията за многократна употреба

1. **Софтуерното инженерство – какво е софтуер, видове софтуер, същност и обхват на софтуерното инженерство.**

Софтуерът представлява **инструкции**, които казват на компютъра какво да прави и при изпълнение осигуряват **желан резултат**, който е свързан с **удовлетворяване на характеристики, функционалнсти, производителност, сигурност** и други. В софтуера се включват и **структури от данни**, които дават възможност на програмите адекватно и ефективно да съхраняват и обработват информацията, с която работят. Документите, които описват работата и използването на програмите също са част от софтуера.

Софтуерът е логическа единица, която се **разработва**, а не се произвежда. Той не се износва и не се амортизира в резултат на външни явления. Въпреки това може да подлежи на **остаряване**, тъй като иновациите в тази сфера са често срещано явление в наши дни.

Софтуерът, който е предназначен да се продава на клиент или потребител, се нарича **продукт**. Различават се два типа софтуерни продукти:

– **общи (generic)** – самостоятелни продукти/системи, които се продават на пазара и са предназначени за всеки потребител, който иска да си ги закупи;

– **поръчани (customized)** – продукти/системи, които са възложени от отделен клиент и се разработват специално за неговите нужди.

Изискванията към първия тип продукти се определят от самата организация, която ги разработва и в повечето случай са породени от или създават някаква нужда на пазара, а във втория – от клиента поръчител.

Видове софтуер:

- **Системен софтуер** - съвкупност от програми, предназначени да **обслужват други програми**. Характеризират се с интензивна комуникация с хардуера и разрешават синхронизационни проблеми при работа с множество процеси. Пример за такъв софтуер са операционните системи, драйверите за устройства, компилатори и др.
- **Приложен софтуер** - програми, които решават специфична нужда на бизнеса или потребителите. Това са например, текстови редактори или редактори на изображения, медийни плейъри и други

- **Научен софтуер** - обслужва нуждите на отделните дялове на науката. Използва се за симулации на системи, за емпирично тестване, анализи, автоматизирано проектиране и други.
- **Вграден (embedded) софтуер** - вгражда се в различни от класическия компютър хардуерни устройства – телефони, прахосмукачки, перални, микровълнови фурни, автомобили, климатици и много други. Може да е насочен както към потребителски, така и към бизнес нужди.
- **Уеб приложения** - в най-простата си форма са множество свързани файлове хипертекст. Съвременните уеб приложения обикновено са многослойни и могат да се използват от множество потребители едновременно.
- **Изкуствен интелект** - използва нечислови алгоритми за решаване на сложни проблеми, които не подлежат на директен анализ. Това са например приложения за разпознаване на образи, звук или почерк. Невронни мрежи, линейна регресия и други.
- **Наследен (legacy) софтуер** - софтуер, който е разработен преди няколко десетилетия и **продължава да се използва като непрекъснато е бил модифициран**, за да задоволи промени в бизнес изисквания, регулации или платформи (хардуерни и софтуерни). Ключовите характеристики на наследения софтуер са **дълъг живот и критична важност за бизнеса**.

Определение: Софтуерното инженерство е дисциплина, която се занимава с **проектирането, разработката, поддръжката и управлението** на софтуерни системи с високо качество, при спазване на **систематични, дисциплинирани и измерими подходи**.

Основни аспекти:
- **Процес на разработка**: Методи и модели за управление на жизнения цикъл на софтуера.
- **Технологии и инструменти**: Езици за програмиране, рамки и платформи за разработка.
- **Управление на проекти**: Планиране, мониторинг и контрол на софтуерни проекти.
- **Качество на софтуера**: Тестване, верификация и валидация.

2. Софтуерен процес – фази и основни дейности, видове модели и езици за моделиране, шаблони за описание.

Софтуерният процес е:
- **Последователност от предвидими стъпки** – план, който следваме, за да създадем продукт или система в срок и с високо качество;
- **Рамка на задачите**, които е необходимо да се изпълнят, за да се изгради висококачествен софтуер;
- **Процесът е повече от процедура** – **процесът е съвкупност от процедури**, организирани така, че да се изграждат продукти, задоволяващи определени цели и стандарти.

Етапи при разработването на софтуер:
- **Изисквания**: Събиране и анализ на изискванията.
- **Проектиране**: Архитектурно и детайлно проектиране на системата.

- **Реализация**: Програмиране на системата.
- **Тестване**: Unit testing, интеграционно и системно тестване.
- **Интеграция и внедряване**: Инсталиране и конфигурация на софтуера.
- **Поддръжка**: Корекции, актуализации и подобрения.

Основни дейности – малък брой, които са приложими за всички софтуерни проекти:
- **Комуникация** – интензивно взаимодействие и сътрудничество с клиента. Събиране и разбиране на изисквания за функционалността и на ограниченията върху работата и разработката на продукта;
- **Планиране** – създава се план за бъдещата работа по разработка на софтуера. Описват се рисковете, необходимите ресурси, работните продукти, които ще се произведат и времевия график;
- **Моделиране** – включва две действия:
    - **Анализ**, в резултат на което се създават като работни продукти модел на анализа и спецификация на изискванията;
    - **Проектиране/дизайн** – определя се дизайна на данните, архитектурата, интерфейса и на ниво компоненти. Работните продукти са модел на дизайна и спецификация на дизайна.
- **Конструиране** – включва генериране на код (ръчно или автоматично) и последователното тестване – на самостоятелни компоненти, на модулите, на подсистемите и на цялата система, а също и потребителско тестване (бета-тестване);
- **Внедряване** – софтуерът се предоставя на клиента и той го оценява

Модел на софтуерен процес и видове модели:
- **(Workflow model) Модел на потока от дейности** – описва последователност от дейности, които се извършват в процеса на разработка. Отделните дейности се описват със своите входни и изходни параметри;
- **(Dataflow model) Модел на потока от данни** – проследява преобразуването на данните и при какви дейности става това;
- **(Role/action model) Модел на роля/действие –** описва ролите в екипа и дейностите, за които всяка роля отговаря.

Софтуерните модели на процеси могат да са **описателни** – описват как се разработва софтуерна система или **предписателни** – предписват как би трябвало да се разработи нова софтуерна система.

Езици за моделиране – видове (UML)
Езиците за моделиране се използват за детайлно и изчерпателно представяне на:
– дейностите, които трябва да се извършат;
– ролите на хората;
– структурата и същността на артефактите, които се създават и/или поддържат;
– средствата, които се използват

Спрямо организацията си, езиците се разделят на:
– **entity-relation –** същности и връзки между тях;
– **role-interaction** – роли и техните взаимоотношения;
– **object-oriented** – основната единица е обект, който има данни и

функционалност.

В зависимост от формалността си, езиците се разделят на

- **Формални**
- **Полуформални**
- **Неформални**

Шаблони:
Шаблон – описание на общо решение на общ проблем или въпрос, на базата на което може да се извлече детайлно решение на специфичен проблем.
Шаблон на процес – шаблон, който описва доказан, успешен подход или последователност от действия за разработване на софтуер. Описва какво трябва да се направи, а не точни детайли как трябва да се направи. Шаблоните могат да бъдат дефинирани на различни нива на абстракция.

Всеки шаблон трябва да съдържа:
– **Начален контекст;**
– Проблем;
– Решение;
– Краен контекст;
– Свързани шаблони;
– Начини на употреба

3. **Сравнителен анализ на описателни модели на софтуерен процес – модел на водопада, прототипен модел, модел на бързата разработка, спираловиден модел и др.**
    a. **Модел на водопада**

Най-старият метод на структурирано разработване на софтуер. Предлага систематизиран и последователен подход. Всяка дейност трябва да бъде **напълно завършена** (след одобряване на множество документи) **преди да премине към следващата** и след това не трябва да се връщаме към тази дейност отново. **Линеен последователен подход.**

Трудно е за потребителя да формулира **всички изисквания в началото. Клиентът** трябва да е **търпелив**, тъй като не получава обратна връзка как се разработва системата, преди тя да е завършена. **Трудно адаптиране към промени** в изискванията и **късно се намират проблеми**.

Подходящ, когато изискванията са **осъзнати и ясно формулирани още в началото** и когато са ясно дефинирани ролите на всеки. Може да се приложи и за по-големи проекти, но при които времето и бюджета не са критични

    b. **Модел на бързата разработка (RAD - Rapid App Development)**

високоскоростна адаптация на модела на водопада. Целта му е кратък цикъл на разработка (60-90 дни). Следват се основните дейности както при модела на водопада с разликата, че при „планиране“ се разделя проектът на модули, така че **различни екипи да моделират и конструират различните модули паралелно**. При конструирането се разчита на предварително съществуващи **софтуерни компоненти**

и автоматично генериране на код. Създадените отделни модули се интегрират заедно при „внедряване“. Бърза разработка чрез итеративни цикли.

Подходящ за проекти, при които могат ясно да се обособят отделни модули. Обикновено се прилага за малки проекти.

c. Прототипен модел

i. **Еволюционен прототип** - представя базов вариант на системата, който после се доразвива до крайния вариант на системата

ii. **Изхвърлен прототип (throw-away prototype)** - създава се прототип, предлагащ ограничено подмножество от функциите на крайната система, който се представя на клиента с цел определяне и валидиране на изискванията.

Може да използва значителни ресурси, а да не успее да удовлетвори очакванията. Използва **допълнителни ресурси** за създаване и поддържане на прототипа. Прототипът може да доведе до лошо проектирана система, ако самият той стане част от крайния продукт. Моделът **не е** подходящ за разработване на софтуерни системи, където проблемът е добре разбран и интерфейсът е ясен и прост.
Подходящ за проекти, при които не са достатъчно ясни потребителските изисквания и дизайнът на софтуерната система.

d. Спираловиден модел

еволюционен модел на софтуерен процес, който съчетава прототипния модел и модела на водопада, като се поставя **акцент на анализ на риска**. Моделът е **скъп за прилагане**. **Сложен за управление и изпълнение.** Изисква опитни екипи и добро планиране. Изисква се участието на разработчици с компетентност за оценка на рисковете.

Прилага се за големи проекти. Благодарение на итерациите е гъвкав и **промени в изискванията не са толкова голям проблем както при модела на водопада**
например.

e. Agile

4. Концепцията за многократна употреба

Многократна употреба: Практика за използване на **съществуващи компоненти, код, библиотеки и рамки** в нови разработки, с цел **спестяване на време и ресурси**. Предимства:

- **Спестяване на време и усилия** (По-ефективно използване на човешкия труд;)
- **Намалява времето за разработка и тестване.**
- **Подобрено качество:** Използването на вече проверени и тествани компоненти намалява вероятността от грешки.
- **Консистентност:** Осигурява последователност в разработките и улеснява поддръжката.
- **Намаляване на риска от провал на проекта**

Сред недостатъците са:

- **цената за поддръжка може да е повече;**
- **споделена отговорност;**
- **трудност при намирането на подходящи компоненти.**

Съществува баланс между многократната употреба (reusability) и приложимостта (usability) – **колкото по-обобщен е даден интерфейс, толкова по-вероятно да се използва многократно, но и е толкова по-сложен и труден за употреба.**

Примери за многократна употреба:

- **COTS – Commercial Off-The-Shelf:** COTS системите обиконвени са цели приложения, които предлагат **API (Application Programming Interface** – приложно програмен интерфейс). Приложими са в широк кръг от области. Липса на контрол върху развитието и появата на нови версии на продуктите; (Пример: API за Image recognition; API за превалутиране; Stripe - payment API)
- **Компонентно-базирана разработка** - Изграждане на софтуер от многократно използваеми компоненти.

Компонентите могат да осъществяват връзка помежду си единствено чрез входно-изходни точки, наречени интерфейси.
– **Входен (requires) интерфейс** – дефинира какво трябва да се предостави на компонента, така че той да се изпълни според спецификацията;
– **Изходен (provides) интерфейс** – дефинира услугата (и резултата от нея), която компонентът предоставя, в следствие на изпълнението му при условие, че са удовлетворени изискванията на входния интерфейс

Разработвайки приложението отделяме независимо програмни единици, които наричаме компоненти, които можем да преизползваме на много места. Например ако създаваме web приложение може да направим компонент таблица, което приема данни, които да визуализира и настройки, които определят как да изглежда таблицата и/или как да са форматирани данните. Веднъж разработен този компонент, може да се преизползва на много места в нашето приложение.

- **Библиотеки и рамки (libraries and frameworks)** - Използване на съществуващи софтуерни библиотеки (например: stl в C++) и рамки като React, Angular, Spring

Съществуват **концепции за многократна употреба**, базирани на създаване на програмни единици:

- **Design patterns (Шаблони за проектиране)** - Описва се дадена задача и най-съществените моменти от решението ѝ. Шаблоните са абстрактни и целят да могат да се прилагат в множество различни ситуации, без да зависят от конкретен програмен език. В зависимост от предназначението си, различаваме следните видове шаблони: – **Създаващи (creational)** – използват се за създаване на обекти; – **Структурни (structural)** – използват се за улесняване на създаването на структури от класове; – **Поведенчески (behavioural)** – предоставят методи за комуникация между обектите
- **Генериране на код**. Пример: Генериране на typescript интерфейси от Java класове

- визуално да се отличава нашия текст (булети, подчертавания)

- добра структура
- да дадем нашата интерпретация на нещата
- да преразкажем нещата с наши думи
- да погледнем какво е в лекциите
- класификация на рзличните модели на софтуерни процеси
- базираме на актуалността в практиката
- многократна употреба - да засегнем темата от най-ниската до най-високата абстракция
- Software Engineering 9th Edition by Ian Sommerville (Nasko: summarize with chatGPT)
- подробен конспект на съдържанието (подробно опорни точки)
- какво е това заинтересовано лице
- сравнителен анализ
    - в какви системите се изплзва
    - етапите
    - според големината на приектите
    - продължителност на проекта
- да включим информация за гъвкавите модели